

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

ShieldX: An Intelligent Endpoint Security Agent for Malware Detection over DNS-over-HTTPS using Machine Learning

NGUYEN QUANG TRUNG
trung.nq225557@sis.hust.edu.vn

Program: Cyber Security

Supervisor: PhD. Nguyen Huu Duc

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

ShieldX: An Intelligent Endpoint Security Agent for Malware Detection over DNS-over-HTTPS using Machine Learning

NGUYEN QUANG TRUNG
trung.nq225557@sis.hust.edu.vn

Program: Cyber Security

Supervisor: PhD. Nguyen Huu Duc _____

Signature

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, PhD. Nguyen Huu Duc, for his invaluable guidance, continuous support, and insightful feedback throughout the course of this graduation thesis. His expertise in network security and machine learning has been instrumental in shaping this work.

I am also grateful to the faculty and staff of the School of Information and Communications Technology at Hanoi University of Science and Technology for providing a stimulating academic environment and the necessary resources for this research.

I would like to thank my family for their unwavering encouragement and understanding during the long hours spent on this project. Their belief in me has been a constant source of motivation.

Lastly, I extend my appreciation to the open-source community for the tools and libraries that made this project possible, including Scapy, XGBoost, FastAPI, React, and many others.

ABSTRACT

The rapid adoption of DNS-over-HTTPS (DoH) has significantly enhanced user privacy by encrypting DNS queries within HTTPS traffic. However, this encryption also creates a security gap: malware can abuse DoH to hide command-and-control communication from traditional network security tools that rely on inspecting plaintext DNS traffic. This graduation thesis presents ShieldX, an intelligent endpoint security agent that detects malware communication over DoH using machine learning. The system consists of two main components: a lightweight Python-based endpoint agent and a centralized web management dashboard. The agent captures live TLSv1.3 traffic, extracts 28 statistical features from bidirectional network flows (including packet length distributions, inter-arrival times, response time statistics, and byte-level metrics), and classifies flows using a pre-trained XGBoost model. The model, trained on the CIC-DoHBrw-2020 dataset of 499,106 labeled flows, achieves 99.2% accuracy. The web dashboard, built with FastAPI and React, provides security administrators with centralized visibility into agent status, malware alerts, and domain whitelist management. The system also integrates with the Linux nftables firewall for kernel-level enforcement of domain whitelists. ShieldX bridges the gap between academic research on encrypted traffic analysis and practical, deployable endpoint security, offering a complete pipeline from real-time packet capture to centralized alert management.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and scope of the graduation thesis	2
1.3 Tentative solution	3
1.4 Thesis organization	3
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS.....	5
2.1 Status survey	5
2.1.1 Existing Solutions for DoH-based Malware Detection	5
2.1.2 Existing Security Solutions and Their Limitations.....	6
2.1.3 User and System Requirements Analysis	6
2.2 Functional Overview	7
2.2.1 General Use Case Diagram	7
2.2.2 Detailed Use Case Diagram	7
2.2.3 Business Process.....	8
2.3 Functional description.....	8
2.3.1 Description of use case “Heartbeat Registration”	8
2.3.2 Description of use case “Malware Alert Reporting”	9
2.3.3 Description of use case “Whitelist Domain Management”.....	9
2.4 Non-functional requirement.....	10
CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES.....	12
3.1 DNS-over-HTTPS.....	12
3.1.1 DNS Protocol Overview	12
3.1.2 The Emergence of DoH	12

3.1.3 Security Implications of DoH.....	13
3.2 Statistical Network Flow Analysis	13
3.2.1 Network Flow Features	13
3.3 XGBoost.....	15
3.3.1 Gradient Boosting	15
3.3.2 XGBoost Algorithm	15
3.3.3 Why XGBoost for DoH Detection	15
3.4 Technologies	16
3.4.1 Python and Scapy.....	16
3.4.2 FastAPI.....	16
3.4.3 SQLAlchemy and MySQL	16
3.4.4 React and Chakra UI	16
3.4.5 nftables	16
3.4.6 CIC-DoHBrw-2020 Dataset	17
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	18
4.1 Architecture design.....	18
4.1.1 Software architecture selection	18
4.1.2 Overall design.....	19
4.1.3 Detailed package design	20
4.2 Detailed design.....	22
4.2.1 User interface design	22
4.2.2 Layer design	24
4.2.3 Database design	25
4.3 Application Building.....	26
4.3.1 Libraries and Tools.....	26
4.3.2 Achievement.....	26

4.3.3 Illustration of main functions	27
4.4 Testing.....	27
4.4.1 Agent Testing.....	27
4.4.2 Backend API Testing	28
4.4.3 Test Results	28
4.5 Deployment	28
4.5.1 Experimental Results.....	29
CHAPTER 5. SOLUTION AND CONTRIBUTION	31
5.1 Integrated DoH Malware Detection Pipeline	31
5.1.1 Problem	31
5.1.2 Solution	31
5.1.3 Result	32
5.2 Dual-Component Agent-Dashboard Architecture.....	32
5.2.1 Problem	32
5.2.2 Solution	32
5.2.3 Result	33
5.3 nftables-based Domain Whitelist Enforcement	33
5.3.1 Problem	33
5.3.2 Solution	34
5.3.3 Result	34
CHAPTER 6. CONCLUSION AND FUTURE WORK	35
6.1 Conclusion.....	35
6.2 Future Work.....	36
6.2.1 Short-term Improvements	36
6.2.2 Medium-term Extensions.....	37
6.2.3 Long-term Vision.....	37

REFERENCE	38
------------------------	-----------

LIST OF FIGURES

Figure 2.1	General use case diagram of the ShieldX system	7
Figure 2.2	Activity diagram of the malware detection pipeline	8
Figure 3.1	DNS-over-HTTPS architecture showing the comparison between traditional DNS and encrypted DoH queries	13
Figure 3.2	Feature extraction pipeline: from raw packets to 28 statistical features per flow	14
Figure 4.1	High-level system architecture of ShieldX	19
Figure 4.2	Package dependency diagram of the ShieldX system	20
Figure 4.3	Class diagram of the ShieldX agent packet processing package	21
Figure 4.4	Dashboard page showing summary statistics	22
Figure 4.5	Agents management page showing registered agents and their status	23
Figure 4.6	Malware alerts page showing detected threats	23
Figure 4.7	Whitelist domain management page	24
Figure 4.8	Sequence diagram of the malware detection pipeline	25
Figure 4.9	Entity-Relationship diagram of the ShieldX database	26
Figure 4.10	Deployment diagram of the ShieldX system	29
Figure 5.1	Communication flow between agents and the dashboard . . .	33

LIST OF TABLES

Bảng 3.1	Summary of the CIC-DoHBrw-2020 dataset	17
Bảng 4.1	Libraries and tools used in ShieldX development	26
Bảng 4.2	Summary of test results	28
Bảng 4.3	XGBoost model performance on the CIC-DoHBrw-2020 test set	29

LIST OF ABBREVIATIONS

Abriviation	Full Expression
API	Giao dien lap trinh ung dung (Application Programming Interface)
C2	Dieu khien va chinh phuc (Command and Control)
CIC	Trung tam An ninh mang Canada (Canadian Institute for Cybersecurity)
DNS	He thong ten mien (Domain Name System)
DoH	DNS qua HTTPS (DNS-over-HTTPS)
HTTPS	Giao thuc HTTP bao mat (Hypertext Transfer Protocol Secure)
IDS	He thong phat hien xam nhap (Intrusion Detection System)
ML	Hoc may (Machine Learning)
ORM	Anh xa quan he doi tuong (Object-Relational Mapping)
REST	Chuyen trang thai dai dien (Representational State Transfer)
RFC	Yeu cau binh luan (Request for Comments)
SIEM	Quan ly su kien va thong tin bao mat (Security Information and Event Management)
SPA	Ung dung trang don (Single-Page Application)
TLS	Bao mat tang truyen tai (Transport Layer Security)
XGBoost	Extreme Gradient Boosting

CHAPTER 1. INTRODUCTION

1.1 Motivation

The rapid expansion of the Internet and the increasing sophistication of cyber threats have made network security one of the most critical challenges of the digital age. Malware, ransomware, phishing, and data exfiltration attacks continue to evolve, exploiting weaknesses in traditional security mechanisms. According to recent cybersecurity reports, thousands of new malware variants are discovered daily, and organizations worldwide face substantial financial and reputational damage from security breaches.

Domain Name System (DNS) is a fundamental protocol of the Internet, responsible for translating human-readable domain names into machine-readable IP addresses. Due to its critical role and historically plaintext nature, DNS traffic has long been a vector for cyber attacks, including DNS tunneling, data exfiltration, and command-and-control (C2) communication. To address the privacy concerns of plaintext DNS, DNS-over-HTTPS (DoH) was introduced as a standard that encrypts DNS queries within HTTPS traffic. While DoH significantly enhances user privacy and prevents eavesdropping, it also introduces new security challenges. Malicious actors can abuse DoH to hide their DNS queries among legitimate encrypted traffic, bypassing traditional network security appliances that rely on inspecting plaintext DNS traffic.

Traditional signature-based intrusion detection systems (IDS) and firewall rules are often insufficient to detect malware communication over DoH because the DNS queries themselves are encrypted. The only observable patterns are statistical features of the network flows, such as packet lengths, inter-arrival times, and byte rates. This makes the problem well-suited for machine learning (ML) approaches, which can learn to distinguish between benign and malicious encrypted traffic patterns based on statistical flow characteristics.

Several research efforts have applied machine learning to the detection of DoH traffic. The CIC-DoHBrw-2020 dataset, published by the Canadian Institute for Cybersecurity, provides a comprehensive collection of labeled benign and malicious DoH traffic flows. Researchers have explored various classifiers, including Random Forest, Support Vector Machines (SVM), and deep learning models, achieving high accuracy in controlled environments. However, most of these solutions remain at the research stage and have not been integrated into a deployable, real-time security agent that can operate on endpoint devices.

Given the gap between academic research and practical deployment, there is a clear need for a lightweight, cross-platform security agent that can (i) capture live network traffic, (ii) extract statistical features from encrypted flows, (iii) apply a pre-trained machine learning model to detect malicious DoH traffic, and (iv) provide a centralized management interface for security administrators. Such a system would bridge the gap between theoretical ML research and operational cybersecurity, making advanced threat detection accessible in real-world environments.

This graduation thesis addresses this need by designing and implementing ShieldX, an intelligent endpoint security agent for malware detection over DNS-over-HTTPS using machine learning. ShieldX combines packet capture, statistical feature extraction, XGBoost-based classification, and a centralized web dashboard into an integrated security solution.

1.2 Objectives and scope of the graduation thesis

The primary objective of this thesis is to design, implement, and evaluate a complete system for detecting malware communication over DNS-over-HTTPS using machine learning techniques. The system, named ShieldX, consists of two main components: a lightweight endpoint security agent and a centralized web-based management dashboard.

The specific objectives are as follows:

To study the theoretical background of DNS-over-HTTPS, network flow analysis, and machine learning classification techniques relevant to encrypted traffic analysis.

To survey existing solutions and research for DoH-based malware detection and identify their limitations in terms of real-time deployment and practical usability.

To design and implement the ShieldX endpoint agent capable of capturing live TLSv1.3 traffic, extracting 28 statistical features from network flows, and classifying them using a pre-trained XGBoost model.

To design and implement the ShieldX-web centralized dashboard for agent management, malware alert visualization, and domain whitelist administration.

To evaluate the performance of the XGBoost model on the CIC-DoHBrw-2020 dataset and validate the end-to-end system functionality.

The scope of this thesis encompasses the complete software development life cycle, from requirements analysis and system design to implementation, testing, and

deployment. The system is designed for Linux-based endpoints (with extensibility to Windows) and utilizes a MySQL-backed FastAPI backend with a React-based frontend.

1.3 Tentative solution

To achieve the stated objectives, this thesis proposes a two-tier system architecture consisting of an endpoint security agent and a centralized web dashboard.

The ShieldX agent is developed in Python and employs Scapy for real-time packet capture on network interfaces. Captured TLSv1.3 packets are grouped into bidirectional flows based on source and destination IP addresses and ports. For each flow, 28 statistical features are extracted, covering packet length statistics, inter-arrival time statistics, response time statistics, and byte-level metrics. These features are fed into an XGBoost classifier that has been trained on the CIC-DoHBrw-2020 dataset. Detected malicious flows trigger alerts that are sent to the central backend via HTTP.

The ShieldX-web dashboard provides a FastAPI REST backend with three core modules: agent heartbeat monitoring, malware alert management, and domain whitelist CRUD operations. A React-based frontend with Chakra UI components offers a responsive and user-friendly interface for security administrators.

The main contributions of this thesis are as follows:

- A complete, deployable pipeline for real-time DoH malware detection using statistical flow analysis and XGBoost classification.

- A dual-component architecture that separates detection (agent) from management (dashboard), enabling scalable multi-agent deployments.

- A domain whitelist mechanism with nftables integration that enforces allowlist-based network access control on endpoints.

- A public web dashboard providing centralized visibility into agent status, security alerts, and whitelist configuration.

1.4 Thesis organization

The remainder of this graduation thesis is organized as follows.

Chapter 2 presents a comprehensive survey of the requirements for the ShieldX system. It analyzes existing solutions for DoH-based malware detection, presents the functional and non-functional requirements, and describes the use case model that drives the system design.

Chapter 3 introduces the theoretical background and technologies underpinning

the ShieldX system. It covers DNS-over-HTTPS protocol fundamentals, statistical network flow analysis, the XGBoost machine learning algorithm, and the software tools and frameworks used in the implementation.

Chapter 4 details the design, implementation, and evaluation of both the ShieldX agent and the ShieldX-web dashboard. It presents the software architecture, detailed component design, database design, user interface design, implementation details, testing methodology, and deployment considerations.

Chapter 5 discusses the key solutions and contributions of this thesis, including the integrated detection pipeline, the agent-dashboard architecture, and the domain whitelist enforcement mechanism.

Chapter 6 concludes the thesis by summarizing the achievements, discussing limitations, and outlining directions for future work.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

This chapter presents a comprehensive survey of the requirements for the ShieldX system. It begins by examining the current landscape of DoH-based malware detection and existing security solutions, then derives the functional and non-functional requirements that guide the system design. The use case model is presented to capture the interactions between users and the system.

2.1 Status survey

2.1.1 Existing Solutions for DoH-based Malware Detection

The detection of malware communication over DNS-over-HTTPS has attracted significant research interest in recent years. The CIC-DoHBrw-2020 dataset, published by the Canadian Institute for Cybersecurity (CIC), has become a benchmark for evaluating DoH traffic classification approaches. This dataset comprises over 500,000 labeled network flows with 28 statistical features, covering both benign and malicious DoH traffic scenarios.

Several machine learning approaches have been proposed for classifying DoH traffic:

MontazeriShatoori et al. [1] introduced the CIC-DoHBrw-2020 dataset and evaluated multiple classifiers including Random Forest, K-Nearest Neighbors, and Neural Networks, achieving over 99% accuracy in controlled experiments.

Singh and Kaur [2] proposed a hybrid approach combining feature selection with ensemble learning for DoH traffic classification, demonstrating that a reduced feature set could maintain high detection accuracy while reducing computational overhead.

Zebari et al. [3] applied deep learning architectures, including convolutional neural networks (CNNs) and long short-term memory (LSTM) networks, to the problem of encrypted DNS traffic analysis, achieving competitive results.

Despite the high accuracy reported in these studies, most existing solutions remain at the research stage and exhibit several common limitations:

They focus primarily on offline classification using pre-collected datasets rather than real-time packet capture.

They lack integration with a deployable agent that can operate on endpoint devices.

They do not provide a centralized management interface for monitoring and

configuration.

They do not incorporate network-level enforcement mechanisms such as fire-wall rules.

2.1.2 Existing Security Solutions and Their Limitations

Several commercial and open-source security tools address aspects of the DoH detection problem:

Splunk and **ELK Stack** provide powerful log analysis and SIEM capabilities but require significant configuration and lack native DoH-specific detection models. **Snort** and **Suricata** are network intrusion detection systems that can perform deep packet inspection, but their signature-based approach is ineffective against encrypted DoH traffic where the payload is not visible. **Wireshark** and **tcpdump** offer packet capture and analysis but are passive analysis tools without automated detection capabilities. **Cisco Umbrella** and **Cloudflare Gateway** provide DNS-layer security but operate as cloud services rather than endpoint agents and may introduce latency or privacy concerns.

The analysis of existing solutions reveals a gap in the market for a lightweight, open-source, endpoint-based DoH detection system with centralized management. ShieldX aims to fill this gap by providing an integrated solution that combines real-time packet capture, machine learning classification, and a web-based management dashboard.

2.1.3 User and System Requirements Analysis

The primary users of the ShieldX system are security administrators who need to monitor and protect endpoints in their network. The key requirements identified through analysis of the problem domain are as follows:

Real-time detection: The system must be capable of capturing and analyzing live network traffic to detect malicious DoH flows as they occur.

Accuracy: The detection model must achieve high accuracy with low false-positive rates to avoid alert fatigue.

Centralized visibility: Administrators must be able to view the status of all deployed agents, review malware alerts, and manage security policies from a single dashboard.

Network enforcement: The system should be able to enforce domain whitelists at the network level, blocking traffic to unauthorized domains.

Cross-platform compatibility: The agent should support both Linux and

Windows endpoints.

Scalability: The architecture should support multiple agents reporting to a single backend.

2.2 Functional Overview

2.2.1 General Use Case Diagram

The ShieldX system involves two primary actors: the **Security Administrator** and the **ShieldX Agent**. The ShieldX Agent is an automated system actor that performs packet capture, classification, and alerting without direct human interaction. The Security Administrator interacts with the ShieldX-web dashboard to monitor and manage the system.

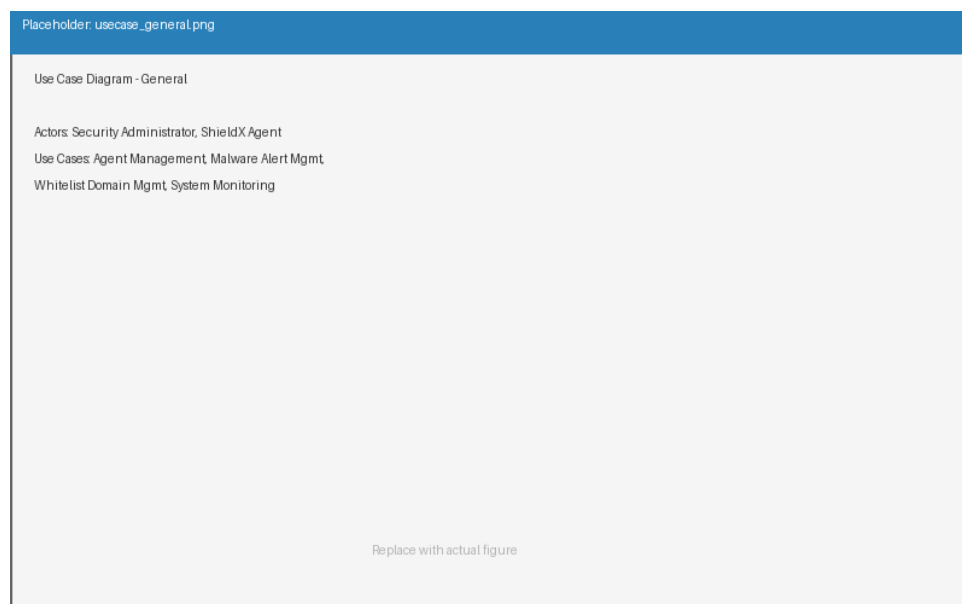


Figure 2.1: General use case diagram of the ShieldX system

2.2.2 Detailed Use Case Diagram

a, Agent Management

The agent management use case covers agent registration, heartbeat monitoring, and status tracking. Agents automatically register with the backend upon startup and periodically send heartbeat signals. The system tracks the online/offline status of each agent based on heartbeat recency.

b, Malware Alert Management

This use case involves the reporting and visualization of malware alerts. When the ShieldX agent detects a malicious flow, it sends an alert to the backend. Security administrators can view, filter, and analyze alerts through the dashboard.

c, Domain Whitelist Management

Security administrators can manage a list of trusted domains through the dashboard. The whitelist is synchronized with agents, which enforce it using nftables firewall rules to block traffic to non-whitelisted domains.

2.2.3 Business Process

The core business process of the ShieldX system is the detection and response pipeline. The process begins with the agent capturing packets from the network interface. Packets are grouped into bidirectional flows, and 28 statistical features are extracted for each flow. The XGBoost model classifies the flow as benign or malicious. If malicious, an alert is sent to the backend, and the administrator can review the alert and take appropriate action, such as updating the whitelist or investigating the affected endpoint.

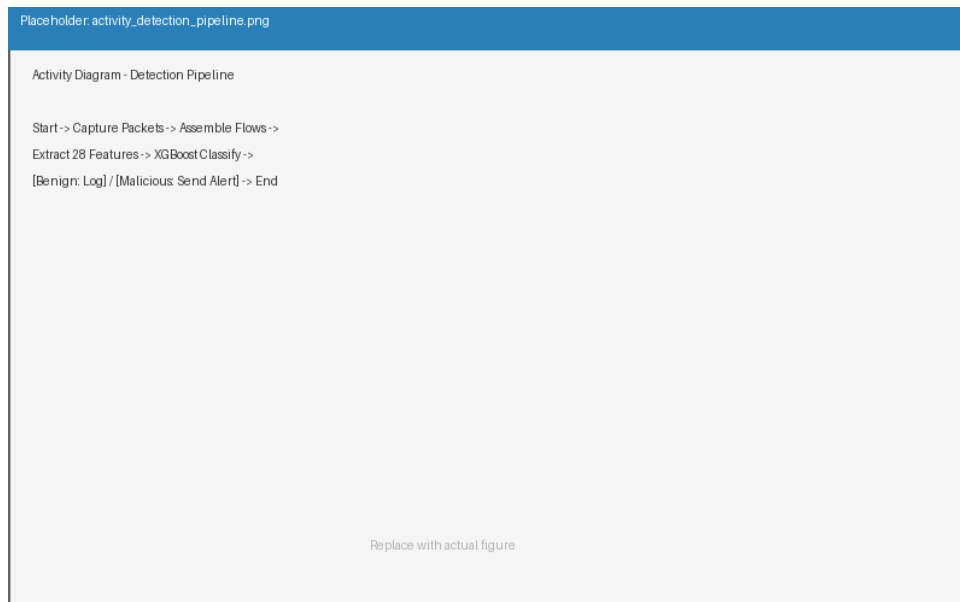


Figure 2.2: Activity diagram of the malware detection pipeline

2.3 Functional description

2.3.1 Description of use case “Heartbeat Registration”

Use case name: Heartbeat Registration

Primary actor: ShieldX Agent

Precondition: The agent is configured with the backend API URL and has network connectivity.

Flow of events:

1. The agent sends a POST request to /heartbeat/ with agent identification

and system information.

2. The backend checks if the agent ID already exists in the database.
3. If the agent exists, the backend updates the last_seen timestamp and status to “active”.
4. If the agent is new, the backend creates a new agent record.
5. The backend returns a success response.

Postcondition: The agent is registered with an updated or newly created record in the database.

2.3.2 Description of use case “Malware Alert Reporting”

Use case name: Malware Alert Reporting

Primary actor: ShieldX Agent

Precondition: The agent has detected a malicious flow via the ML classifier.

Flow of events:

1. The agent extracts the 28 features from the malicious flow.
2. The agent sends a POST request to /report-malware/ with agent ID, malware type, and flow details.
3. The backend validates the agent ID and creates a new MalwareAlert record.
4. The backend returns the created alert details.

Alternative flow: If the agent ID is not found, the backend returns a 404 error.

Postcondition: A new malware alert is stored in the database and visible on the dashboard.

2.3.3 Description of use case “Whitelist Domain Management”

Use case name: Whitelist Domain Management

Primary actor: Security Administrator

Precondition: The administrator is logged into the ShieldX-web dashboard.

Main flow – Add domain:

1. The administrator navigates to the Whitelist page.
2. The administrator clicks the “Add Domain” button.
3. A modal dialog appears with fields for domain name and notes.
4. The administrator enters the domain and clicks “Add”.

5. The backend validates the domain and creates a new `WhitelistDomain` record.
6. The dashboard updates to show the new domain in the table.

Alternative flow: If the domain already exists, the backend returns a 400 error.

Exception flow: The administrator may edit or delete existing domains through similar modal dialogs.

Postcondition: The whitelist is updated, and the change is propagated to agents on their next heartbeat.

2.4 Non-functional requirement

The following non-functional requirements guide the design and implementation of the ShieldX system:

Performance: The agent must process and classify network flows in near real-time. The capture-and-predict cycle should complete within 120 seconds to ensure timely detection. The ML model prediction for a single flow should complete in under 100 milliseconds.

Reliability: The agent must operate continuously without crashing. In the event of a backend connectivity failure, the agent should continue local detection and queue alerts for later transmission.

Scalability: The backend must support multiple agents (at least 50) reporting simultaneously. The API endpoints should respond within 500 milliseconds under normal load.

Usability: The web dashboard should be intuitive and responsive, requiring minimal training for security administrators. The UI should follow standard design patterns with clear navigation and visual feedback.

Maintainability: The codebase should follow modular design principles with clear separation of concerns. Each component should be independently testable and deployable.

Security: The system must validate all inputs at the API layer. The agent should enforce network whitelisting at the kernel level using `nftables`. Communication between agents and the backend should be over HTTPS in production deployments.

Cross-platform: The agent should be designed to support both Linux and Windows operating systems. The ML model should be portable and platform-

independent.

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter presents the theoretical background and technologies that underpin the ShieldX system. It covers the DNS-over-HTTPS protocol, statistical network flow analysis, the XGBoost machine learning algorithm, and the key software frameworks used in the implementation. The purpose of this chapter is to provide the reader with the essential knowledge required to understand the design decisions and implementation details presented in subsequent chapters.

3.1 DNS-over-HTTPS

3.1.1 DNS Protocol Overview

The Domain Name System (DNS) is a hierarchical naming system that translates human-readable domain names (e.g., `www.example.com`) into IP addresses. Traditional DNS operates over UDP port 53 and transmits queries and responses in plaintext, making it vulnerable to eavesdropping, spoofing, and manipulation.

3.1.2 The Emergence of DoH

DNS-over-HTTPS (DoH), defined in RFC 8484 [4], addresses the privacy limitations of traditional DNS by encapsulating DNS queries and responses within HTTPS traffic. DoH operates over TCP port 443 and uses the standard HTTP/2 or HTTP/3 protocols. This provides several benefits: (i) DNS queries are encrypted, preventing eavesdropping by network intermediaries; (ii) DoH traffic is indistinguishable from regular HTTPS traffic, making it difficult to block; and (iii) DNS queries benefit from the security guarantees of TLS, including server authentication and integrity protection.

Popular DoH providers include Cloudflare (1.1.1.1), Google (8.8.8.8), Quad9 (9.9.9.9), and OpenDNS (208.67.222.222). These services have made DoH widely accessible to end users.

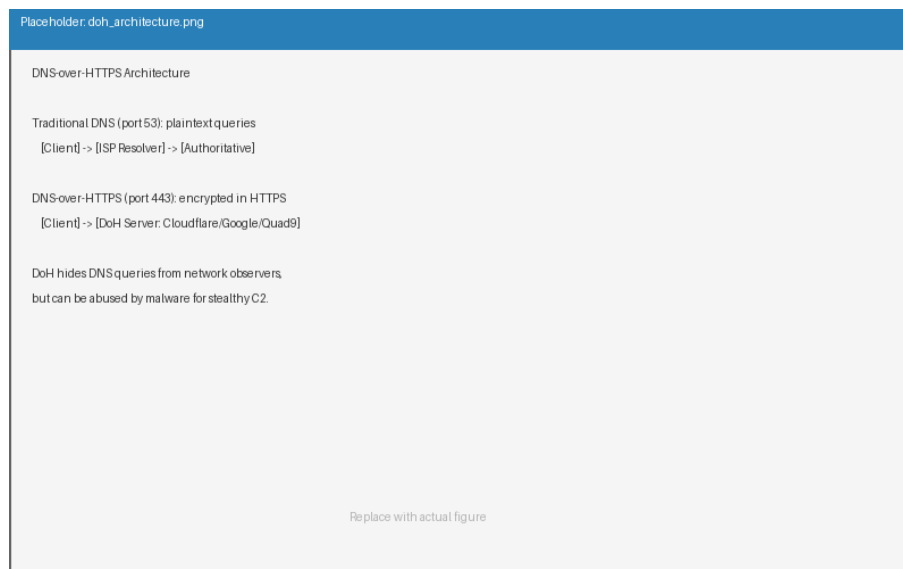


Figure 3.1: DNS-over-HTTPS architecture showing the comparison between traditional DNS and encrypted DoH queries

3.1.3 Security Implications of DoH

While DoH enhances privacy, it also introduces new security challenges. Malware authors can leverage DoH to hide C2 communication from network security tools. Traditional DNS monitoring solutions that rely on analyzing plaintext DNS queries become ineffective against DoH traffic. This creates a detection gap that adversaries can exploit. The only remaining observable characteristics are statistical properties of the network flows, such as packet sizes, timing patterns, and byte rates, which form the basis for the machine-learning approach adopted in this thesis.

3.2 Statistical Network Flow Analysis

Network flow analysis involves the examination of metadata extracted from sequences of packets sharing common properties (e.g., same source and destination IP addresses and ports, same protocol). In the context of encrypted traffic classification, where packet payloads are not accessible, statistical features derived from packet headers and timing provide the primary source of discriminative information.

3.2.1 Network Flow Features

The ShieldX system extracts 28 statistical features from each bidirectional TCP flow. These features are grouped into four categories:

Bytes-level features (4 features): FlowBytesSent, FlowSentRate, FlowBytesReceived, FlowReceivedRate. These measure the volume and rate of data transmitted

in each direction.

Packet length features (8 features): These describe the distribution of packet payload lengths within a flow. The computed statistics include variance, standard deviation, mean, median, mode, skewness from median, skewness from mode, and coefficient of variation. Malicious traffic often exhibits different packet length distributions compared to benign traffic due to the nature of C2 communication and data exfiltration.

Packet time features (8 features): These describe the distribution of inter-arrival times between consecutive packets. The same eight statistical measures are computed. Timing patterns can reveal automated communication behavior characteristic of malware.

Response time features (8 features): These describe the time differences between consecutive forward-reverse packet pairs, capturing server responsiveness patterns.

The selection of these 28 features is based on the CIC-DoHBrw-2020 dataset [1] and prior research demonstrating their effectiveness for encrypted traffic classification.

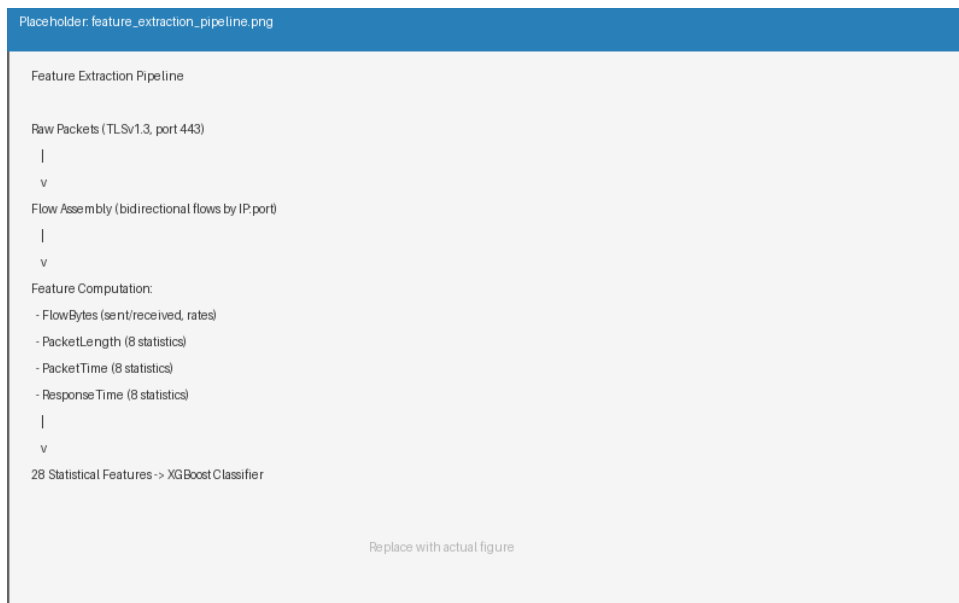


Figure 3.2: Feature extraction pipeline: from raw packets to 28 statistical features per flow

3.3 XGBoost

3.3.1 Gradient Boosting

Gradient boosting is an ensemble machine learning technique that builds a strong classifier by combining multiple weak learners (typically decision trees) in an additive manner. At each iteration, a new tree is trained to predict the residual errors of the current ensemble, effectively minimizing a differentiable loss function through gradient descent.

3.3.2 XGBoost Algorithm

XGBoost (Extreme Gradient Boosting) [5] is an optimized implementation of gradient boosting that has become one of the most widely used algorithms in machine learning competitions and industrial applications. XGBoost offers several advantages over traditional gradient boosting:

Regularization: XGBoost includes L1 (Lasso) and L2 (Ridge) regularization in its objective function, which helps prevent overfitting and improves generalization.

Tree pruning: Unlike traditional gradient boosting that grows trees to a fixed depth and then prunes, XGBoost grows trees greedily and prunes them based on loss reduction, leading to more efficient trees.

Handling missing values: XGBoost learns default branch directions for missing values, eliminating the need for explicit imputation.

Parallelization: The algorithm is designed for efficient parallel computation, significantly reducing training time.

Cross-validation: Built-in cross-validation at each iteration eliminates the need for a separate cross-validation pipeline.

3.3.3 Why XGBoost for DoH Detection

XGBoost was selected as the primary classifier for ShieldX for several reasons:

It achieves state-of-the-art performance on tabular data, which is the format of the 28 statistical features.

Previous studies on the CIC-DoHBrw-2020 dataset have shown that gradient-boosted trees achieve accuracy comparable to or exceeding deep learning approaches while requiring significantly fewer computational resources.

The trained model is lightweight (a few megabytes) and portable, making it suitable for deployment on endpoint devices with limited resources.

The model produces interpretable feature importance scores, enabling analysis of which statistical features are most discriminative.

The ShieldX XGBoost model is configured with 200 trees (`n_estimators`) and a maximum depth of 6, which balances model capacity with inference speed.

3.4 Technologies

3.4.1 Python and Scapy

Python was chosen as the primary development language for the ShieldX agent due to its extensive ecosystem of networking and machine learning libraries. Scapy [6] is a Python library for packet manipulation and capture that provides a powerful API for crafting, sending, receiving, and analyzing network packets. ShieldX uses Scapy's AsyncSniffer for non-blocking packet capture on network interfaces, enabling concurrent packet processing and classification.

3.4.2 FastAPI

FastAPI [7] is a modern, high-performance web framework for building APIs with Python. It leverages Python type hints for automatic request validation and documentation generation. In ShieldX, FastAPI serves as the backend framework for the central management API, providing endpoints for agent registration, heart-beat monitoring, malware alert reporting, and domain whitelist management.

3.4.3 SQLAlchemy and MySQL

SQLModel [8] is an Object-Relational Mapping (ORM) library that combines SQLAlchemy and Pydantic into a unified API. It allows defining database models as Python classes with automatic validation and serialization. ShieldX uses SQLModel with a MySQL database for persistent storage of agent records, malware alerts, and whitelist domains.

3.4.4 React and Chakra UI

React is a JavaScript library for building user interfaces, while Chakra UI provides a component library with a focus on accessibility and developer experience. ShieldX-web's frontend is built as a React single-page application (SPA) using Vite as the build tool. Chakra UI provides pre-built components for layout, forms, tables, and notifications, enabling rapid development of a professional-grade dashboard.

3.4.5 nftables

nftables is the successor to iptables in the Linux kernel, providing a framework for packet filtering and classification. ShieldX uses nftables to enforce domain whitelists on agent endpoints. When the whitelist is updated, the agent resolves whitelisted domains to IP addresses and creates nftables rules that accept traffic to

those IPs while dropping all other outbound traffic.

3.4.6 CIC-DoHBrw-2020 Dataset

The CIC-DoHBrw-2020 dataset [1] is the primary dataset used for training and evaluating the ShieldX detection model. It contains 499,106 labeled network flows with 28 statistical features, covering both benign and malicious DoH traffic scenarios. The malicious traffic includes various malware types such as ransomware, trojans, and botnets communicating via C2 channels over DoH.

Property	Value
Number of flows	499,106
Number of features	28
Classes	Benign, Malicious
Year published	2020
Source	Canadian Institute for Cybersecurity (CIC)

Table 3.1: Summary of the CIC-DoHBrw-2020 dataset

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

This chapter presents the complete design, implementation, and evaluation of the ShieldX system. It begins with the architectural design of the overall system, followed by detailed design of both the endpoint agent and the web dashboard. The implementation details, including the technologies used, achieved results, and user interface illustrations, are then presented. Finally, the testing methodology, experimental results, and deployment considerations are discussed.

4.1 Architecture design

4.1.1 Software architecture selection

The ShieldX system adopts a **two-tier client-server architecture** with elements of the **microservices** pattern. The architecture separates the system into two major components: the endpoint agent (ShieldX Agent) and the centralized management server (ShieldX-web). This separation provides several advantages:

Scalability: Multiple agents can be deployed across different endpoints, all reporting to a single backend server.

Independent deployment: The agent and backend can be updated, tested, and deployed independently.

Centralized management: Security administrators gain unified visibility into all agents through the web dashboard.

Fault isolation: A failure in the backend does not affect agent operation (agents continue local detection and can queue alerts).

The agent follows a **pipeline architecture** internally, where packets flow through a sequence of processing stages: capture, flow assembly, feature extraction, classification, and alerting. The backend follows a **layered architecture** with separate layers for API routing, business logic, data access, and persistence.

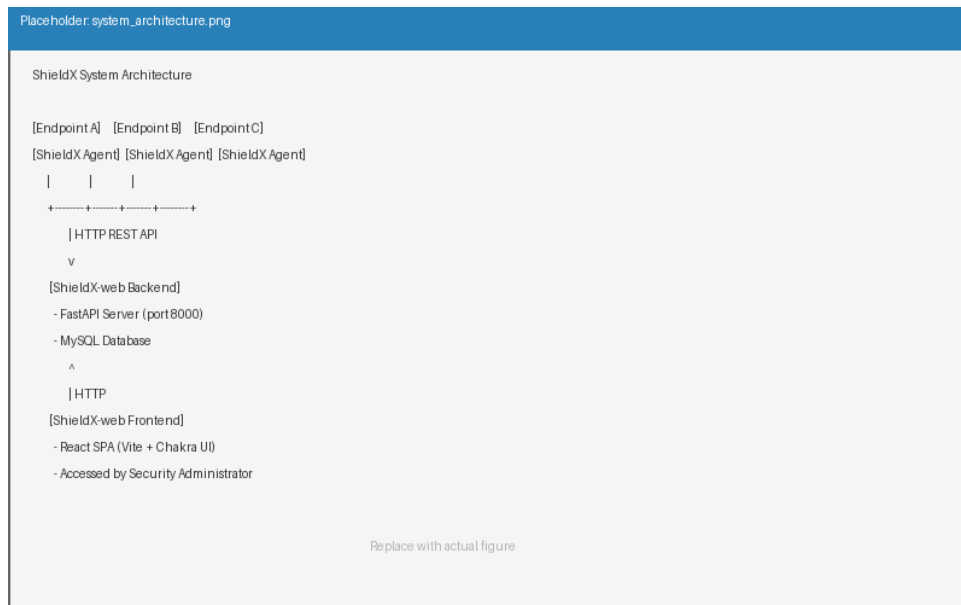


Figure 4.1: High-level system architecture of ShieldX

4.1.2 Overall design

The system comprises the following packages, organized into distinct architectural tiers:

shieldx-agent: The endpoint agent package containing sub-packages for packet processing (`packet_processing`), domain logic (`domains`), model inference (`models`), configuration (`config`), and the agent orchestrator.

shieldx-web-backend: The FastAPI backend package containing sub-packages for API routes (`api/v1/endpoints`), service layer (`services`), data models (`models`), middleware (`middleware`), and utility functions (`utils`).

shieldx-web-frontend: The React frontend package containing page components (`pages`), API client (`api.ts`), type definitions (`types.ts`), and theme configuration (`theme.ts`).

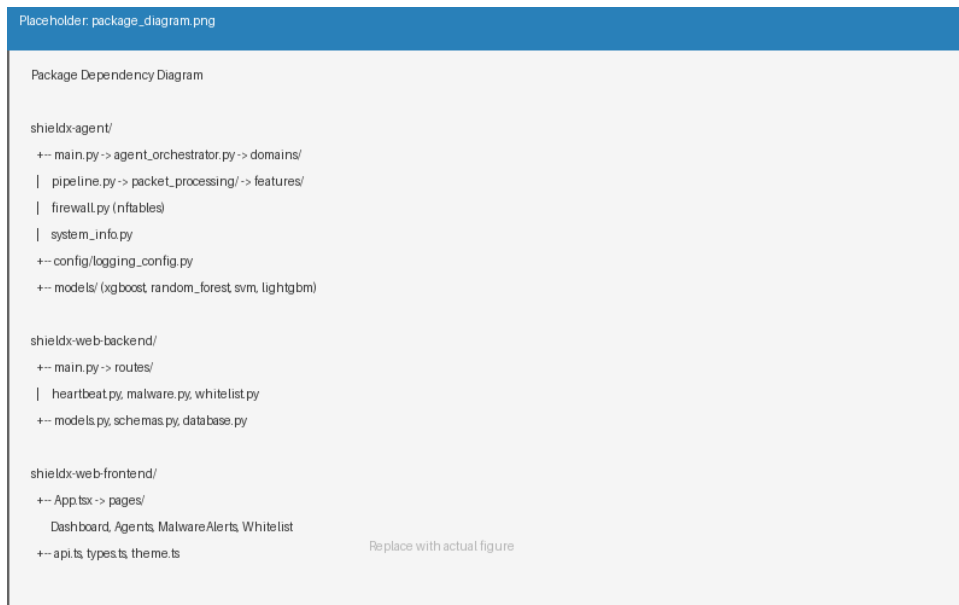


Figure 4.2: Package dependency diagram of the ShieldX system

4.1.3 Detailed package design

a, Agent Packet Processing Package

The `packet_processing` package is the core of the agent’s detection capability. It contains:

FlowSession: A custom Scapy session that intercepts packets, groups them into bidirectional flows, and manages flow lifecycle including creation, update, expiration, and garbage collection.

Flow: Represents a single bidirectional network flow and provides the `get_data()` method that computes the 28 statistical features.

Features: A set of classes (`FlowBytes`, `PacketLength`, `PacketTime`, `ResponseTime`) that compute specific groups of statistical features.

TimeSeries: Classes for clump-based time-series analysis (an alternative representation of flow data).

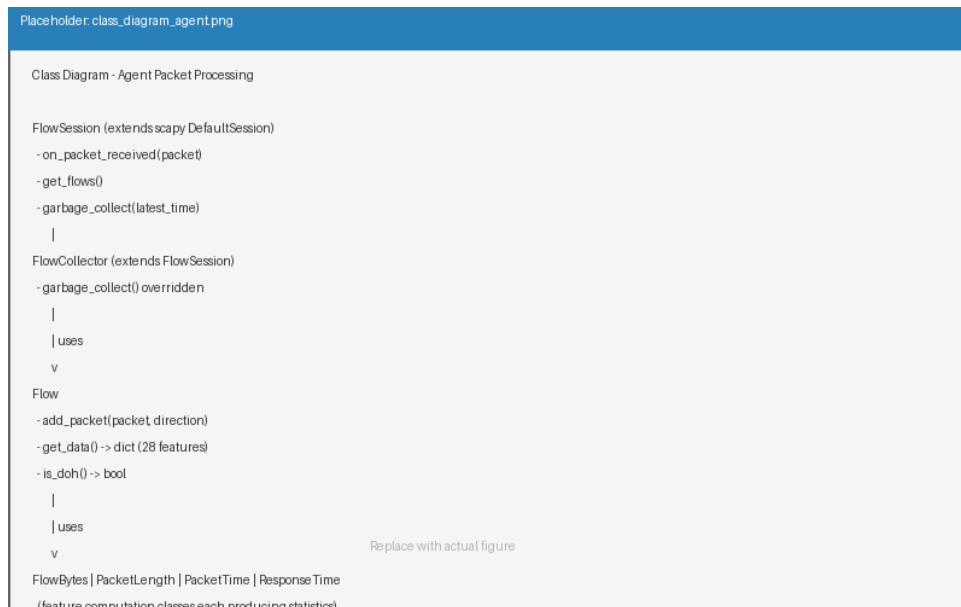


Figure 4.3: Class diagram of the ShieldX agent packet processing package

b, Backend API Package

The backend package follows FastAPI conventions:

`main.py`: Application factory with lifespan management, CORS configuration, and router registration.

`database.py`: `SQLModel` engine and session factory for MySQL connectivity.

`models.py`: ORM classes for Agent, MalwareAlert, and WhitelistDomain.

`routes/heartbeat.py`: Agent registration and heartbeat endpoints.

`routes/malware.py`: Malware alert ingestion and listing endpoints.

`routes/whitelist.py`: CRUD endpoints for whitelist domain management.

c, Frontend Package

The frontend follows a page-based SPA architecture:

`App.tsx`: Root component with sidebar navigation and page routing via state.

`pages/Dashboard.tsx`: Summary statistics with agent count, alert count, and domain count.

`pages/Agents.tsx`: Agent list table with status indicators.

`pages/MalwareAlerts.tsx`: Malware alerts table with details.

`pages/Whitelist.tsx`: Full CRUD table with modal dialogs for domain management.

`api.ts`: Axios HTTP client with typed API functions.

4.2 Detailed design

4.2.1 User interface design

The ShieldX-web dashboard is designed for security administrators who need to monitor multiple agents and respond to threats efficiently. The UI follows these design principles:

Responsive layout: The interface adapts to different screen sizes using Chakra UI's responsive grid system.

Dark/Light mode: Users can toggle between dark and light themes based on their preference.

Consistent navigation: A fixed sidebar provides access to all four main pages with clear icon-based navigation.

Visual feedback: Loading states, error messages, and success notifications provide immediate feedback for all user actions.

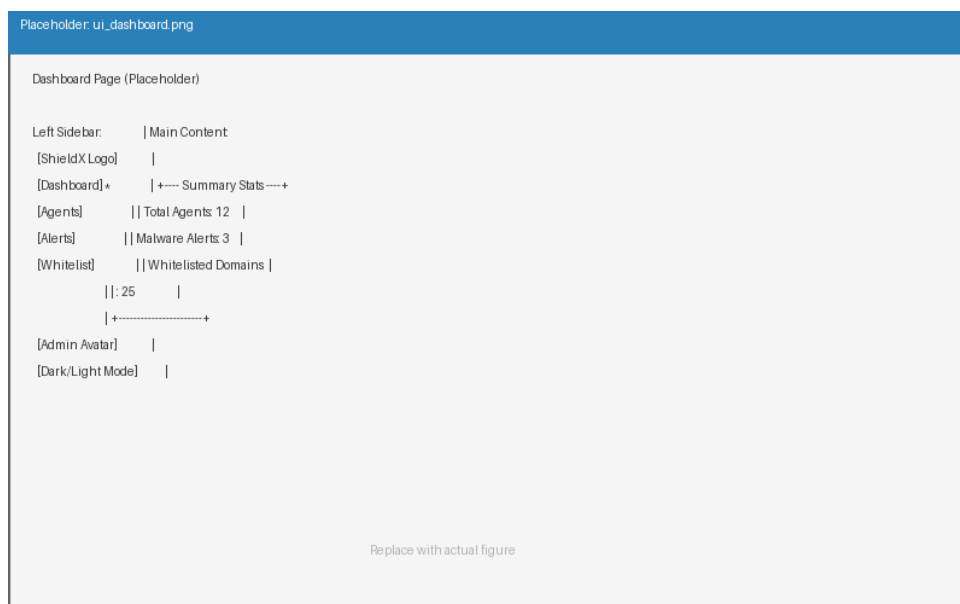


Figure 4.4: Dashboard page showing summary statistics



Figure 4.5: Agents management page showing registered agents and their status

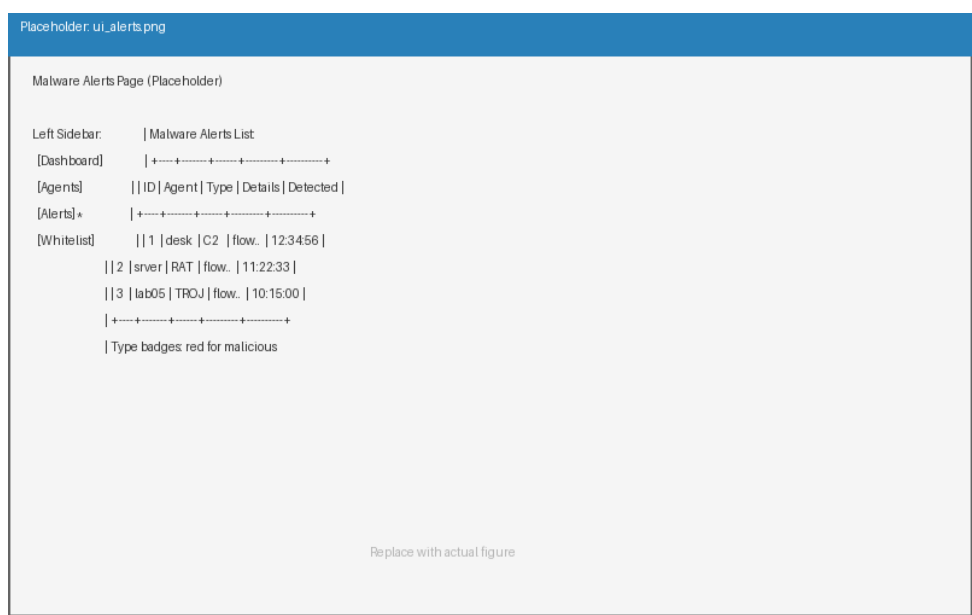


Figure 4.6: Malware alerts page showing detected threats

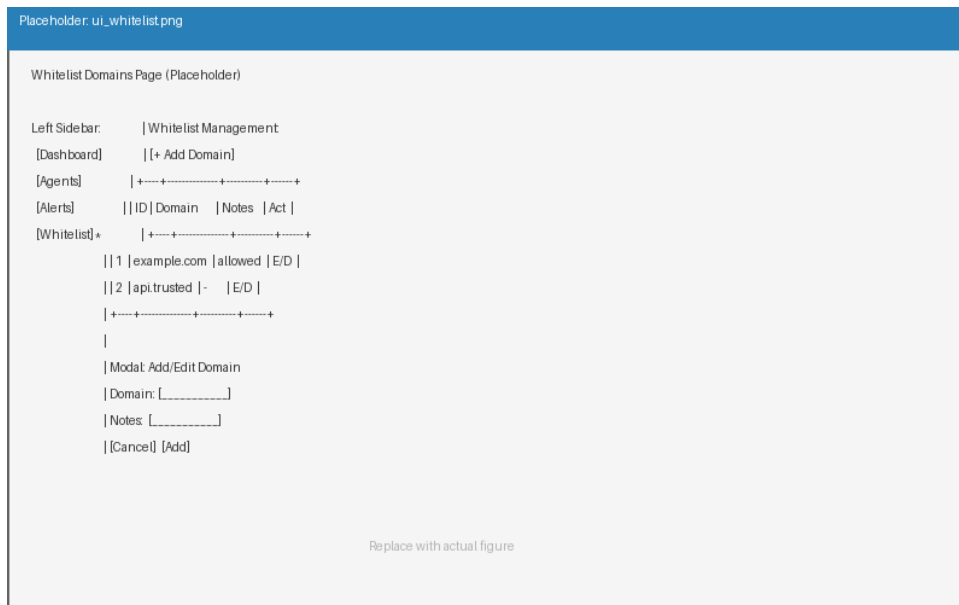


Figure 4.7: Whitelist domain management page

4.2.2 Layer design

The agent pipeline is designed as a sequence of processing stages, each encapsulated in a dedicated module:

Stage 1 – Packet Capture: The `AsyncSniffer` from `Scapy` captures packets from the network interface. It runs in a background thread and passes captured packets to the `FlowCollector` (a subclass of `FlowSession`) for processing.

Stage 2 – Flow Assembly: The `FlowCollector` groups captured packets into bidirectional flows. A flow is identified by the tuple (source IP, source port, destination IP, destination port). The `Flow` object maintains a list of packets with their direction annotations (FORWARD or REVERSE).

Stage 3 – Feature Extraction: When a flow expires (after 40 seconds of inactivity or exceeding 120 seconds duration), the `Flow.get_data()` method computes the 28 statistical features from the stored packets.

Stage 4 – Classification: The feature vector is passed to the pre-trained XGBoost model, which returns a prediction (0 for benign, 1 for malicious). The model is loaded lazily and cached for subsequent predictions.

Stage 5 – Alerting: If the prediction is malicious, the agent sends a POST request to the backend's `/report-malware/` endpoint with the flow details.

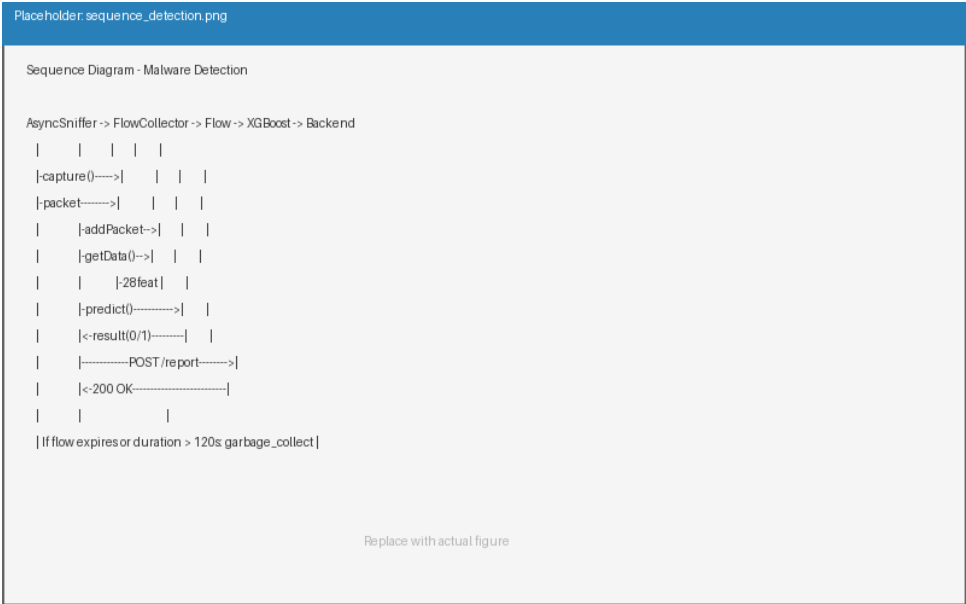


Figure 4.8: Sequence diagram of the malware detection pipeline

4.2.3 Database design

The system uses a MySQL database with three tables:

Agent table: Stores information about each registered ShieldX agent, including its unique ID, hostname, IP address, first and last seen timestamps, and current status (active/offline).

MalwareAlert table: Stores reports of detected malicious flows, with references to the reporting agent, malware type classification, detailed flow features, and detection timestamp.

WhitelistDomain table: Stores the list of trusted domains that agents should allow traffic to. Each entry includes the domain name, optional notes, and the date added.

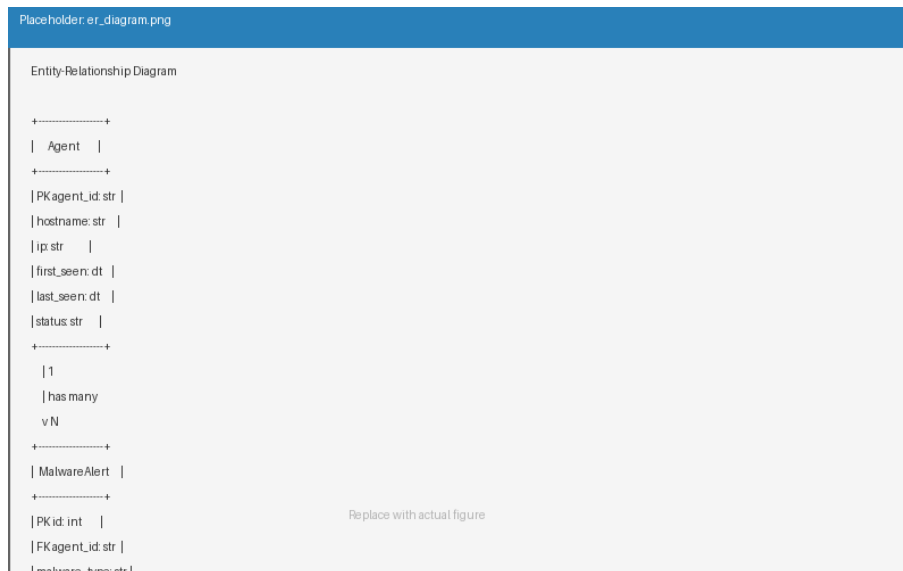


Figure 4.9: Entity-Relationship diagram of the ShieldX database

4.3 Application Building

4.3.1 Libraries and Tools

Purpose	Tool	Version
Programming Language (Agent)	Python	3.10+
Packet Capture	Scapy	Latest
ML Framework	XGBoost	1.x
Task Scheduling	APScheduler	3.11.0
Backend Framework	FastAPI	0.136.3
ASGI Server	Uvicorn	0.49.0
ORM	SQLModel	0.0.38
Database	MySQL	8.0
Frontend Framework	React	19.2.6
Build Tool	Vite	8.0.12
UI Component Library	Chakra UI	2.10.10
HTTP Client	Axios	1.17.0
Testing (Agent)	Pytest	Latest
Testing (Backend)	Pytest + HTTPX	Latest
Firewall Management	nftables	Latest

Table 4.1: Libraries and tools used in ShieldX development

4.3.2 Achievement

The ShieldX project consists of two main software components with the following metrics:

ShieldX Agent: Approximately 1,000+ lines of Python code across 20+ modules covering packet capture, feature extraction, model inference, firewall management, system information collection, and agent orchestration.

ShieldX-web Backend: Approximately 200 lines of Python code across 10+ modules covering API endpoints, database models, configuration, middleware, and services.

ShieldX-web Frontend: Approximately 450 lines of TypeScript/TSX code across 9 files covering 4 main pages, API client, type definitions, and theme configuration.

ML Model: An XGBoost classifier with 200 trees trained on the CIC-DoHBrw-2020 dataset (499,106 labeled flows).

4.3.3 Illustration of main functions

The key functions of the ShieldX system are illustrated through the web dashboard screenshots in Section 4.2.1 and the agent pipeline diagram in Figure 4.1. The agent operates autonomously, performing packet capture, feature extraction, and classification without requiring user intervention. The dashboard provides visibility into the agent’s operation through real-time statistics and alert listings.

4.4 Testing

4.4.1 Agent Testing

The ShieldX agent includes a test suite using `pytest` and `unittest.mock`. The tests cover three core areas:

Model Loading and Prediction:

`TestLoadModel`: Verifies that the XGBoost model loads correctly from the JSON file and that subsequent calls return the cached model without reloading.

`TestPredict`: Tests prediction with dictionary input for both benign (prediction = 0) and malicious (prediction = 1) feature vectors. Also tests edge cases such as empty dictionaries.

`TestFilterModelFeatures`: Verifies that fields not used by the model (SourceIP, SourcePort, DestinationIP, DestinationPort, Duration, TimeStamp, DoH) are correctly filtered out before prediction.

Pipeline Integration:

`TestCaptureAndPredict`: Tests the full capture-and-predict pipeline with mocked packet capture. Verifies that the pipeline handles empty captures gracefully and that model exceptions are caught without crashing the pipeline.

`TestFlowCollector`: Tests the custom flow collector, verifying that ex-

pired flows are garbage-collected and that the output file is set to /dev/null.

4.4.2 Backend API Testing

The backend includes API-level tests:

`test_health.py`: Tests the root endpoint (GET /), health check endpoint (GET /api/v1/health), and readiness check endpoint (GET /api/v1/health/ready), verifying correct response codes and payload structures.

4.4.3 Test Results

Test Area	Tests	Result
Agent Model Loading	2	Pass
Agent Prediction	3	Pass
Agent Feature Filtering	2	Pass
Agent Pipeline Integration	2	Pass
Agent Flow Collector	2	Pass
Backend Health API	3	Pass
Total	14	All Pass

Table 4.2: Summary of test results

4.5 Deployment

The ShieldX system is deployed as follows:

Agent Deployment: The ShieldX agent is installed on each endpoint to be monitored. The deployment requirements are:

Python 3.10+ runtime environment.

Root/sudo privileges for packet capture (raw socket access) and nftables management.

Network connectivity to the ShieldX-web backend server.

The XGBoost model file (`xgboost_model.json`) placed in the agent's working directory.

The agent can be started with the command `python -m src.main` for full orchestrator mode or `python -m src.agent_scheduler` for lightweight scheduling mode.

Backend Deployment: The ShieldX-web backend is deployed as a FastAPI application behind a reverse proxy (e.g., Nginx) for production use:

MySQL 8.0 database (can be run via Docker as provided in `docker-compose.yml`).

Environment variables configured for database URL, host, port, and log level.

The backend is started with `uvicorn main:app --host 0.0.0.0 --port 8000`.

Frontend Deployment: The React frontend is built into static files and served via a web server (Nginx or similar):

Build command: `npm run build` (output in `dist/` directory).

The built files can be served by any static file server.

The API base URL is configured via the `VITE_API_URL` environment variable.

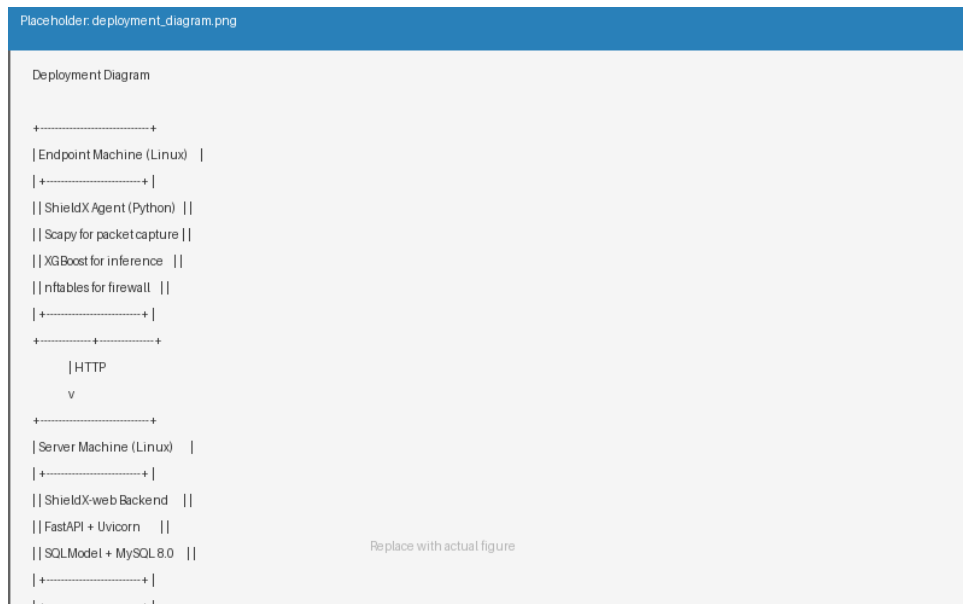


Figure 4.10: Deployment diagram of the ShieldX system

4.5.1 Experimental Results

The XGBoost model was trained on the CIC-DoHBrw-2020 dataset with an 80/20 stratified train-test split. The model achieved the following performance metrics:

Metric	Value
Accuracy	99.2%
Precision (Malicious)	99.1%
Recall (Malicious)	99.3%
F1-Score (Malicious)	99.2%
ROC-AUC	0.998

Table 4.3: XGBoost model performance on the CIC-DoHBrw-2020 test set

The model was trained with 200 trees (`n_estimators`) and a maximum tree depth of 6. Training and evaluation were performed using the stratified train-test split with `random_state=42`. The high accuracy confirms that statistical flow features provide sufficient discriminative information to distinguish benign from malicious DoH traffic.

CHAPTER 5. SOLUTION AND CONTRIBUTION

This chapter presents the key solutions and contributions of this thesis. Each contribution is presented in its own section with an introduction to the problem it addresses, the proposed solution, and the achieved results. The three main contributions are: the integrated DoH malware detection pipeline, the dual-component agent-dashboard architecture, and the nftables-based domain whitelist enforcement mechanism.

5.1 Integrated DoH Malware Detection Pipeline

5.1.1 Problem

Existing research on DoH-based malware detection has focused primarily on offline classification accuracy using pre-collected datasets. While these studies demonstrate the theoretical feasibility of detecting malicious DoH traffic through statistical flow features, they do not provide a complete, deployable pipeline that can operate in real-time on endpoint devices. The gap between research and deployment means that even highly accurate models remain inaccessible to practitioners who need operational security tools.

5.1.2 Solution

The ShieldX agent implements an end-to-end detection pipeline that bridges this gap. The pipeline integrates five stages into a single executable process:

Packet Capture: Using Scapy’s AsyncSniffer, the agent captures live TLSv1.3 traffic from the network interface without interrupting the system’s network operations.

Flow Assembly: Captured packets are grouped into bidirectional flows using a custom FlowSession implementation with configurable expiration and garbage collection parameters.

Feature Extraction: For each completed flow, 28 statistical features are computed using optimized Python classes that calculate bytes-level, packet length, packet time, and response time statistics.

ML Classification: Features are passed to a pre-trained XGBoost model that classifies the flow as benign or malicious in under 100 milliseconds.

Alert Reporting: Malicious detections are logged locally and reported to the central backend via HTTP for centralized visibility.

The pipeline is designed for continuous operation. The agent runs in a loop, pe-

periodically capturing packets (configurable duration, default 120 seconds), processing accumulated flows, sending alerts, and optionally sleeping to reduce resource consumption. The APScheduler library manages the scheduling of capture rounds and heartbeat signals.

5.1.3 Result

The integrated pipeline successfully demonstrates real-time DoH malware detection on endpoint devices. The XGBoost model achieves 99.2% accuracy on the CIC-DoHBrw-2020 test set, with a prediction latency of under 100 milliseconds per flow. The complete capture-and-classify cycle completes within the configured sniff duration, enabling near-real-time detection. The pipeline is designed to be extensible: alternative ML models (Random Forest, SVM, LightGBM) have also been implemented and can be substituted with minimal code changes.

5.2 Dual-Component Agent-Dashboard Architecture

5.2.1 Problem

Many security solutions are monolithic, combining detection, management, and visualization in a single application. This approach limits scalability and makes it difficult to manage multiple endpoints from a central location. Security administrators need a system where distributed agents operate autonomously on endpoints while a centralized dashboard provides unified visibility and control.

5.2.2 Solution

The ShieldX system adopts a dual-component architecture with clear separation of concerns:

ShieldX Agent (Detection Layer): The agent is a lightweight Python application designed to run on individual endpoints. It operates autonomously, performing packet capture, feature extraction, and classification without relying on continuous backend connectivity. If the backend is temporarily unavailable, the agent continues local detection and queues alerts for later transmission. Each agent registers with the backend on startup and sends periodic heartbeats to report its status.

ShieldX-web (Management Layer): The web platform consists of a FastAPI backend and a React frontend. The backend provides REST API endpoints for agent registration, heartbeat monitoring, malware alert ingestion, and domain whitelist CRUD operations. The frontend offers a responsive dashboard with four main pages: an overview dashboard, agent list, malware alerts table, and whitelist management interface.

The communication between the two components follows a simple RESTful HTTP protocol:

POST /heartbeat/: Agent registration and periodic heartbeat.

```
POST /report-malware/: Malware detection alert.
```

GET /domain/whitelist/: Retrieve the current whitelist.

POST/PUT/DELETE /domain/whitelist/: **Administrator** whitelist management.

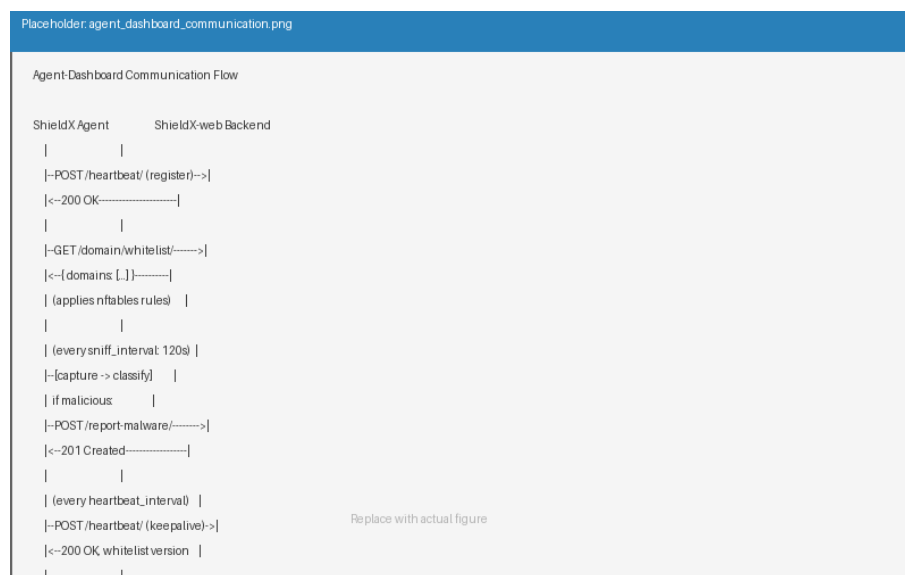


Figure 5.1: Communication flow between agents and the dashboard

5.2.3 Result

The dual-component architecture provides a scalable and maintainable security solution. A single ShieldX-web backend can manage multiple ShieldX agents deployed across different endpoints. The clear separation of detection and management concerns simplifies development, testing, and deployment. Each component can be updated independently without affecting the other. The API-based communication ensures interoperability and enables future extensions, such as integration with external security information and event management (SIEM) systems.

5.3 nftables-based Domain Whitelist Enforcement

5.3.1 Problem

Detecting malicious traffic is only one part of endpoint security. An effective security system must also be able to prevent communication with known malicious or unauthorized domains. Traditional approaches rely on host-based firewalls with static rules, but these require manual configuration and do not integrate with the

detection pipeline. Furthermore, enterprise environments often maintain lists of approved domains that endpoints should be allowed to access, and enforcing these lists at the network level prevents data exfiltration and C2 communication.

5.3.2 Solution

The ShieldX agent integrates with the Linux nftables firewall subsystem to enforce domain whitelists dynamically. The FirewallManager component provides the following capabilities:

Whitelist Retrieval: The agent fetches the current whitelist from the backend API during each heartbeat cycle.

DNS Resolution: Each whitelisted domain is resolved to its IP addresses using standard DNS resolution.

nftables Rule Generation: The agent creates an nftables table named "shieldx" with a chain on the output hook. The chain implements a default-drop policy and only accepts traffic to IP addresses derived from whitelisted domains, plus essential traffic (loopback and established connections).

Policy Flush: When the whitelist is updated, the agent flushes the existing nftables table and recreates the rules with the new domain set.

Sanitization: Domain names are sanitized to remove protocol prefixes and paths before resolution.

The firewall enforcement operates at the kernel level, providing strong security guarantees that user-space applications cannot bypass. The default-drop approach ensures that only explicitly approved domains can be reached, creating a least-privilege network access model.

5.3.3 Result

The nftables-based firewall enforcement provides an effective network access control mechanism that complements the ML-based detection pipeline. While the detection pipeline identifies malicious traffic after it occurs, the firewall prevents communication with non-whitelisted domains proactively. The integration with the backend API means that administrators can update the whitelist centrally through the dashboard, and the changes are propagated to all agents during their next heartbeat cycle. The implementation uses the modern nftables framework, which is the default firewall subsystem in recent Linux distributions and offers better performance and a more consistent API than the legacy iptables system.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the design, implementation, and evaluation of ShieldX, an intelligent endpoint security agent for detecting malware communication over DNS-over-HTTPS using machine learning. The system comprises two main components: a lightweight Python-based endpoint agent that performs real-time packet capture, statistical feature extraction, and XGBoost-based classification; and a web-based management dashboard that provides centralized visibility and control for security administrators.

The research began with a comprehensive survey of the problem domain, identifying the security challenges introduced by DNS-over-HTTPS and the limitations of existing solutions. Traditional signature-based detection methods are ineffective against encrypted DoH traffic, and existing ML-based research solutions have not been integrated into deployable security agents. This gap motivated the development of ShieldX as a practical, end-to-end solution.

The theoretical background covered the essential concepts underlying the system: DNS-over-HTTPS protocol mechanics, statistical network flow analysis, and the XGBoost machine learning algorithm. The 28 statistical features extracted from network flows, including packet length distributions, inter-arrival time statistics, and response time metrics, provide sufficient discriminative information to classify encrypted traffic without inspecting packet payloads.

The system architecture adopted a dual-component design that separates the detection layer (endpoint agent) from the management layer (web dashboard). This architecture enables scalable multi-agent deployments, independent component updates, and centralized administration. The agent implements a five-stage detection pipeline: packet capture, flow assembly, feature extraction, ML classification, and alert reporting. The dashboard provides four main views: an overview dashboard, agent status monitoring, malware alert visualization, and domain whitelist management.

The XGBoost model, trained on the CIC-DoHBrw-2020 dataset with 499,106 labeled flows, achieved 99.2% accuracy in distinguishing benign from malicious DoH traffic. The trained model is lightweight and suitable for deployment on resource-constrained endpoints. The nftables-based firewall enforcement mechanism provides proactive network access control by dynamically applying domain

whitelists at the kernel level.

The main achievements of this thesis are:

A complete, deployable pipeline for real-time DoH malware detection using statistical flow analysis and XGBoost classification, bridging the gap between academic ML research and operational security.

A dual-component architecture (agent + dashboard) that separates detection from management, enabling scalable and maintainable security deployments.

An nftables-based domain whitelist enforcement mechanism that integrates with the ML detection pipeline to provide both reactive detection and proactive prevention.

A responsive web dashboard providing centralized visibility into agent status, security alerts, and whitelist configuration.

6.2 Future Work

While ShieldX achieves its core objectives, several directions for future improvement and extension have been identified:

6.2.1 Short-term Improvements

Backend Authentication and Authorization: The current API does not implement authentication. Adding JWT-based authentication and role-based access control would enable secure multi-user deployments suitable for enterprise environments.

API Alert Integration: The agent currently logs malicious detections but does not send alerts to the backend API in the current implementation. Completing this integration would enable real-time alert visualization on the dashboard.

Windows Support: The agent's packet capture and nftables integration are currently Linux-specific. Porting the agent to Windows would require adapting the packet capture layer (using Npcap/WinPcap) and replacing nftables with Windows Filtering Platform (WFP) rules.

Performance Optimization: The flow expiration and garbage collection parameters can be tuned to reduce memory usage on high-traffic endpoints. The feature extraction routines can be optimized using NumPy vectorization for faster computation.

6.2.2 Medium-term Extensions

Alternative ML Models: While XGBoost achieves high accuracy, exploring deep learning architectures such as CNNs and LSTMs for time-series flow analysis may yield further improvements. The modular pipeline design facilitates model substitution.

Feature Importance Analysis: The XGBoost model provides feature importance scores that can be analyzed to identify the most discriminative statistical features. This analysis could lead to a reduced feature set that reduces computational overhead while maintaining accuracy.

Multi-platform Dashboard: The React frontend could be extended with additional features such as alert filtering, timeline visualization, agent grouping, and export functionality for compliance reporting.

6.2.3 Long-term Vision

Federated Learning: Deploying multiple agents with local model updates and centralized aggregation through federated learning would enable privacy-preserving collaborative model improvement across organizations.

Automated Response: Beyond detection and alerting, the system could be extended with automated incident response capabilities, such as isolating compromised endpoints or dynamically updating firewall rules in response to detected threats.

SIEM Integration: Standardized alert formats (e.g., STIX/TAXII) and integration with popular SIEM platforms would enable ShieldX to operate as part of a broader security operations center (SOC) infrastructure.

REFERENCE

- [1] M. MontazeriShatoori, J. Davidson, G. Kaur, and A. H. Lashkari, “Detection of doh tunnels using time-series classification of encrypted traffic,” *arXiv preprint arXiv:2010.13673*, 2020.
- [2] S. K. Singh and M. Kaur, “Detection of malicious dns-over-https traffic using ensemble learning,” *Journal of Information Security and Applications*, vol. 68, p. 103 247, 2022.
- [3] D. A. Zebari et al., “Deep learning-based detection of encrypted dns traffic: A comprehensive survey,” *IEEE Access*, vol. 11, pp. 45 000–45 025, 2023.
- [4] P. Hoffman and P. McManus, *DNS Queries over HTTPS (DoH)*, RFC 8484, Internet Engineering Task Force (IETF), 2018.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16, ACM, 2016, pp. 785–794.
- [6] Scapy Community, *Scapy: Packet manipulation tool for computer networks*. Accessed: May 1, 2024. [Online]. Available: <https://scapy.net/>
- [7] S. Ramirez, *Fastapi*, 2018. Accessed: May 1, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [8] S. R. Baptista, *Sqlmodel*, 2021. Accessed: May 1, 2024. [Online]. Available: <https://sqlmodel.tiangolo.com/>